Ciclos `for`, Excepções e Módulos

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

Fundamentos de Programação
2026–2027

Baseado em slides de Luís Caires

Outline

1. Da Aula 8 para a Aula 9
2. Ciclos Contados com `for`
3. Tuplos Durante a Iteração
4. Contratos e Excepções
5. Organizar o Programa em Módulos

Da Aula 8 para a Aula 9

Na Aula 8

- construímos listas e dicionários;
- consultámos e alterámos colecções;
- representámos registos com `dataclass`;
- percorremos posições com **`while`**.

Nesta aula

- percorremos colecções directamente com **`for`**;
- combinamos iteração e desempacotamento;
- comunicamos argumentos inválidos com excepções;
- organizamos funções em módulos reutilizáveis.

Quando Conhecemos os Valores a Percorrer

Queremos apresentar todos os números guardados numa lista:

```
1 numeros = [12, 7, 18, 5]
2
3 i = 0
4 while i < len(numeros):
5     print(numeros[i])
6     i += 1
```

Observação

O índice existe apenas para chegar a cada elemento. Quando precisamos dos valores, mas não das posições, Python oferece uma forma mais directa.

Outline

1. Da Aula 8 para a Aula 9
2. Ciclos Contados com `for`
3. Tuplos Durante a Iteração
4. Contratos e Excepções
5. Organizar o Programa em Módulos

Percorrer uma Lista com `for`

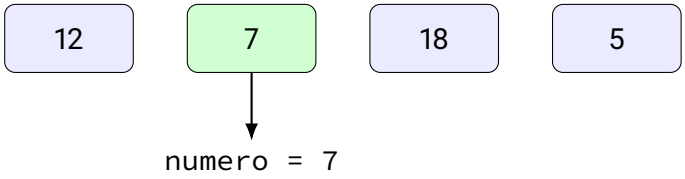
```
1 numeros = [12, 7, 18, 5]
2
3 for numero in numeros:
4     print(numero)
```

Percorrer uma Lista com `for`

```
1 numeros = [12, 7, 18, 5]
2
3 for numero in numeros:
4     print(numero)
```

- Em cada iteração, `numero` recebe o elemento seguinte.
- O ciclo termina depois do último elemento.
- Não temos de criar nem actualizar um índice manualmente.

Como o Ciclo Avança



Modelo de execução

O nome depois de `for` é afectado sucessivamente a cada valor da sequência; o corpo executa uma vez por valor.

`while` ou `for`?

```
1 i = 0
2 while i < len(xs):
3     usa(xs[i])
4     i += 1
```

`while` ou `for`?

```
1 i = 0
2 while i < len(xs):
3     usa(xs[i])
4     i += 1
```

```
1 for x in xs:
2     usa(x)
```

`while` ou `for`?

```
1 i = 0
2 while i < len(xs):
3     usa(xs[i])
4     i += 1
```

```
1 for x in xs:
2     usa(x)
```

- Use `for` para percorrer uma sequência conhecida.
- Use `while` quando a continuação depende de uma condição que muda.
- Se a posição tiver significado, podemos obtê-la explicitamente.

Produzir Inteiros com `range(stop)`

```
1 for i in range(5):
2     print(i)
```

Produzir Inteiros com `range(stop)`

```
1 for i in range(5):
2     print(i)
```

```
0
1
2
3
4
```

Extremo final excluído

`range(5)` produz os inteiros desde `0` até `4`; o valor `5` não pertence à sequência.

Escolher o Início e o Fim

```
1 for i in range(2, 6):
2     print(i)
```

Escolher o Início e o Fim

```
1 for i in range(2, 6):
2     print(i)
```

```
2
3
4
5
```

Leitura de `range(inicio, fim)`

O início é incluído e o fim é excluído. Se `inicio >= fim`, a sequência é vazia quando o passo é positivo.

Controlar o Passo

```
1 for par in range(2, 11, 2):
2     print(par)
```

Controlar o Passo

```
1 for par in range(2, 11, 2):
2     print(par)
```

```
2
4
6
8
10
```

- O terceiro argumento é acrescentado em cada iteração.
- Um passo negativo permite percorrer valores por ordem decrescente.
- O passo nunca pode ser zero.

Acumular Durante um `for`

Somar os valores de uma lista mantém o padrão já conhecido da Aula 5:

```
1 def soma(xs: list[int]) -> int:
2     total = 0
3     for x in xs:
4         total += x
5     return total
```

Acumular Durante um `for`

Somar os valores de uma lista mantém o padrão já conhecido da Aula 5:

```
1 def soma(xs: list[int]) -> int:
2     total = 0
3     for x in xs:
4         total += x
5     return total
```

Invariante

Antes de cada iteração, `total` é a soma dos elementos já percorridos.

Construir uma Nova Colecção

Podemos usar uma lista como acumulador:

```
1 def quadrados(xs: list[int]) -> list[int]:
2     resultado = []
3     for x in xs:
4         resultado.append(x * x)
5     return resultado
```

Construir uma Nova Colecção

Podemos usar uma lista como acumulador:

```
1 def quadrados(xs: list[int]) -> list[int]:
2     resultado = []
3     for x in xs:
4         resultado.append(x * x)
5     return resultado
```

- A lista original não é modificada.
- Cada elemento produz exactamente um valor no resultado.
- O resultado preserva a ordem da entrada.

Previsão: Valores de `range`

Sem executar, indique os valores apresentados e o valor final de `total`:

```
1 total = 0
2 for i in range(1, 8, 2):
3     print(i)
4     total += i
5
6 print(total)
```

Resultado Esperado: Valores de `range`

Problema: percorrer `range(1, 8, 2)` e acumular os seus valores.

Resultado Esperado: Valores de `range`

Problema: percorrer `range(1, 8, 2)` e acumular os seus valores.

```
1
3
5
7
16
```

O ciclo visita quatro inteiros; `8` é o extremo excluído.

Percorrer as Chaves de um Dicionário

```
1 idades = {"Ana": 20, "Bruno": 19}
2
3 for nome in idades:
4     print(nome, idades[nome])
```

Percorrer as Chaves de um Dicionário

```
1 idades = {"Ana": 20, "Bruno": 19}
2
3 for nome in idades:
4     print(nome, idades[nome])
```

- Iterar directamente sobre um dicionário percorre as suas chaves.
- A chave permite consultar o valor associado.
- A ordem é a ordem de inserção das chaves.

Percorrer Chave e Valor com `items`

```
1 idades = {"Ana": 20, "Bruno": 19}
2
3 for nome, idade in idades.items():
4     print(nome, idade)
```

Percorrer Chave e Valor com `items`

```
1 idades = {"Ana": 20, "Bruno": 19}
2
3 for nome, idade in idades.items():
4     print(nome, idade)
```

Cada elemento produzido por `items()` é um tuplo `(chave, valor)`. O cabeçalho do ciclo desempacota imediatamente as duas componentes.

Outline

1. Da Aula 8 para a Aula 9
2. Ciclos Contados com `for`
3. Tuplos Durante a Iteração
4. Contratos e Excepções
5. Organizar o Programa em Módulos

Desempacotar em Cada Iteração

```
1 medicoes = [(1, 18.5), (2, 19.0), (3, 18.7)]
2
3 for instante, valor in medicoes:
4     print(instante, valor)
```

Desempacotar em Cada Iteração

```
1 medicoes = [(1, 18.5), (2, 19.0), (3, 18.7)]
2
3 for instante, valor in medicoes:
4     print(instante, valor)
```

Pré-condição estrutural

Cada elemento da sequência tem de possuir exactamente duas componentes para que o desempacotamento seja possível.

Obter Posição e Valor com `enumerate`

```
1 temperaturas = [18.5, 19.0, 18.7]
2
3 for indice, valor in enumerate(temperaturas):
4     print(indice, valor)
```

Obter Posição e Valor com `enumerate`

```
1 temperaturas = [18.5, 19.0, 18.7]
2
3 for indice, valor in enumerate(temperaturas):
4     print(indice, valor)
```

- **`enumerate`** produz pares `(indice, valor)`.
- O primeiro índice é `0`, salvo indicação em contrário.
- Use-o quando a posição também faz parte do problema.

Percorrer Duas Sequências com `zip`

```
1 nomes = ["Ana", "Bruno", "Carla"]
2 notas = [17, 14, 18]
3
4 for nome, nota in zip(nomes, notas):
5     print(nome, nota)
```

Percorrer Duas Sequências com `zip`

```
1 nomes = ["Ana", "Bruno", "Carla"]
2 notas = [17, 14, 18]
3
4 for nome, nota in zip(nomes, notas):
5     print(nome, nota)
```

Comprimentos diferentes

`zip` termina quando se esgota a sequência mais curta. Se todos os elementos forem necessários, o contrato deve garantir comprimentos iguais.

Previsão: Índice e Desempacotamento

Determine o que é apresentado:

```
1 pontos = [(2, 5), (4, 1), (3, 7)]
2
3 for i, (x, y) in enumerate(pontos, 1):
4     print(i, x + y)
```

Que tuplos são desempacotados em cada iteração?

Resultado Esperado: Índice e Tuplo

Problema: numerar os pontos a partir de `1` e somar as coordenadas.

Resultado Esperado: Índice e Tuplo

Problema: numerar os pontos a partir de `1` e somar as coordenadas.

```
1 7
2 5
3 10
```

`enumerate` produz `(i, ponto)`; o padrão `(x, y)` desempacota depois o ponto.

Outline

1. Da Aula 8 para a Aula 9
2. Ciclos Contados com `for`
3. Tuplos Durante a Iteração
4. Contratos e Excepções
5. Organizar o Programa em Módulos

Quando a Função Não Pode Cumprir o Contrato

Considere uma função que calcula a média de um tuplo de notas.

- O que deve acontecer se o tuplo estiver vazio?
- E se uma nota estiver fora do intervalo `0` a `20`?
- Devolver `0` confundiria um erro com uma média válida.
- Imprimir uma mensagem não informa quem chamou a função.

Excepção

Uma forma explícita de indicar que a operação não conseguiu produzir o resultado prometido pelo seu contrato.

Levantar uma Excepção com `raise`

```
1 def valida_nota(nota: int) -> int:
2     if type(nota) is not int:
3         raise TypeError("a nota deve ser inteira")
4     if nota < 0 or nota > 20:
5         raise ValueError("a nota deve estar entre 0 e 20")
6     return nota
```

Levantar uma Excepção com `raise`

```python
def valida_nota(nota: int) -> int:
    if type(nota) is not int:
        raise TypeError("a nota deve ser inteira")
    if nota < 0 or nota > 20:
        raise ValueError("a nota deve estar entre 0 e 20")
    return nota
```

- `raise` interrompe a execução normal da função.
- O tipo identifica a categoria do problema.
- A mensagem explica a condição violada.

Escolher uma Excepção Informativa

o tipo do argumento não permite a operação.

o tipo é adequado, mas o valor está fora do domínio.

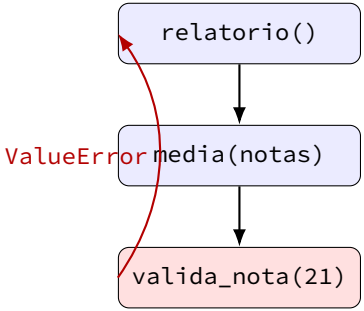
uma posição não existe na sequência.

uma chave não existe no dicionário.

Regra prática

Ao validar explicitamente um contrato, escolha a excepção que melhor descreve a condição violada; não invente um valor de retorno que pareça válido.

Uma Excepção Propaga-se pelas Chamadas



Até ser tratada

Se uma função não tratar a excepção, ela regressa ao ponto onde essa função foi chamada. Sem tratamento, o programa termina e apresenta um traceback.

Ler um Traceback de Baixo para Cima

```
Traceback (most recent call last):
  File "relatorio.py", line 8, in <module>
    print(media((12, 21)))
  File "notas.py", line 12, in media
    total += valida_nota(nota)
  File "notas.py", line 5, in valida_nota
    raise ValueError("nota entre 0 e 20")
ValueError: nota entre 0 e 20
```

Ler um Traceback de Baixo para Cima

```
Traceback (most recent call last):
  File "relatorio.py", line 8, in <module>
    print(media((12, 21)))
  File "notas.py", line 12, in media
    total += valida_nota(nota)
  File "notas.py", line 5, in valida_nota
    raise ValueError("nota entre 0 e 20")
ValueError: nota entre 0 e 20
```

1. Comece pela última linha: tipo e mensagem.
2. Suba para a linha onde a excepção foi levantada.
3. Continue a subir para reconstruir a sequência de chamadas.

Tratar uma Falha Recuperável

```
1 try:
2     nota = valida_nota(valor)
3     print("Nota aceite:", nota)
4 except (TypeError, ValueError) as erro:
5     print("Nota rejeitada:", erro)
```

Tratar uma Falha Recuperável

```
1 try:
2     nota = valida_nota(valor)
3     print("Nota aceite:", nota)
4 except (TypeError, ValueError) as erro:
5     print("Nota rejeitada:", erro)
```

- O bloco `try` contém a operação que pode falhar.
- O bloco `except` descreve uma recuperação concreta.
- Depois do tratamento, a execução continua após o bloco.

Corrigir, Levantar ou Tratar?

Situação	Resposta habitual
índice calculado incorrectamente	corrigir a lógica do programa
argumento fora do contrato	levantar uma excepção na função
entrada inválida do utilizador	tratar junto da leitura e voltar a pedir
excepção sem alternativa segura	deixar propagar

Evitar

Um `except Exception` genérico pode esconder erros de programação que devem ser corrigidos.

Decisão: Que Resposta é Adequada?

Classifique cada situação como corrigir, levantar ou tratar:

1. uma função recebe uma nota `24`, fora do contrato;
2. um ciclo usa `xs[len(xs)]` para aceder ao último elemento;
3. um programa pede um inteiro e o utilizador introduz `"abc"`;
4. uma função interna recebe uma lista que deveria estar ordenada, mas não está.

Justifique onde deve ficar a decisão em cada caso.

Análise: Corrigir, Levantar ou Tratar

Problema: escolher a resposta e o ponto do programa adequados a cada falha.

Análise: Corrigir, Levantar ou Tratar

Problema: escolher a resposta e o ponto do programa adequados a cada falha.

1. A função levanta `ValueError`: o valor não satisfaz o contrato.
2. Corrige-se o índice para `len(xs)- 1`; não se esconde o defeito.
3. Trata-se `ValueError` junto da conversão da entrada.
4. A função levanta uma excepção; o chamador decide se consegue recuperar.

Outline

1. Da Aula 8 para a Aula 9
2. Ciclos Contados com `for`
3. Tuplos Durante a Iteração
4. Contratos e Excepções
5. Organizar o Programa em Módulos

Usar um Módulo da Biblioteca Padrão

```
1 import math
2
3 distancia = math.sqrt(3 * 3 + 4 * 4)
4 print(distancia)  # 5.0
```

Usar um Módulo da Biblioteca Padrão

```
1 import math
2
3 distancia = math.sqrt(3 * 3 + 4 * 4)
4 print(distancia)  # 5.0
```

- `import math` disponibiliza o módulo.
- O nome composto `math.sqrt` mostra de onde vem a função.
- A documentação do módulo define as operações disponíveis.

Construir um Módulo Local

Ficheiro `notas.py`

```
1 def valida_nota(nota: int) -> int:
2     if nota < 0 or nota > 20:
3         raise ValueError("nota
              invalida")
4     return nota
```

Construir um Módulo Local

Ficheiro `notas.py`

```
1 def valida_nota(nota: int) -> int:
2     if nota < 0 or nota > 20:
3         raise ValueError("nota
              invalida")
4     return nota
```

Ficheiro `relatorio.py`

```
1 import notas
2
3 print(notas.valida_nota(17))
```

Construir um Módulo Local

Ficheiro `notas.py`

```
1 def valida_nota(nota: int) -> int:
2     if nota < 0 or nota > 20:
3         raise ValueError("nota
              invalida")
4     return nota
```

Ficheiro `relatorio.py`

```
1 import notas
2
3 print(notas.valida_nota(17))
```

O nome do ficheiro, sem `.py`, torna-se o nome usado por `import`.

Duas Formas de Importar

```
1 import notas
2 notas.valida_nota(17)
```

Duas Formas de Importar

```
1 import notas
2 notas.valida_nota(17)
```

```
1 from notas import valida_nota
2 valida_nota(17)
```

- A primeira forma mantém visível o módulo de origem.
- A segunda introduz directamente os nomes escolhidos.
- Evite `from notas import *`: torna a origem dos nomes ambígua.

Separar Implementação, Cliente e Testes

`notas.py`

Define `valida_nota` e `media`.

`relatorio.py`

Importa as funções e trata a entrada inválida.

`test_notas.py`

Importa as funções e verifica resultados com **`assert`**.

Separar Implementação, Cliente e Testes

notas.py

Define `valida_nota` e `media`.

relatorio.py

Importa as funções e trata a entrada inválida.

test_notas.py

Importa as funções e verifica resultados com **`assert`**.

```
1 from notas import media
2
3 assert media((10, 14, 18)) == 14
```

Cada ficheiro tem uma responsabilidade e reutiliza o mesmo código por importação.

Conclusão

Hoje vimos

- ciclos `for`, `range` e padrões com tuplos
- contratos comunicados com excepções específicas
- módulos para separar implementação, cliente e testes

A seguir

- leitura segura de ficheiros de texto
- interpretação de linhas e tratamento de erros
- escrita de resultados com formatos explícitos